附录
DemandClean：真值受限下任务驱动的数据清洗操作选择

本附录配合论文正文使用。带前缀 A 的编号指本附录，其余节、定理、表与图的编号均指论文正文。

A 定理 1 的证明

Ψ 是多项式: 固定候选的执行顺序。确定性策略是一个向量 $o=(o_1,\dots,o_K)\in\mathcal{O}^K$，环境按顺序执行它，并在只剩 $\lceil\rho n\rceil$ 行时把删除变为不动。记所得状态为 $f(o)=\Phi(o)\in\mathbb{R}^2$。在 π_p 下向量 o 的概率为 $\prod_{i=1}^K p_{o_i}$，因此

$$\Psi(p)=\sum_{o\in\mathcal{O}^K} f(o)\prod_{i=1}^{K} p_{o_i},$$

它是 p 的多项式，因而在紧集 Δ 上连续。在顶点 $p=\mathbf{1}_o$ 处全部概率落在对每个候选施加 o 的策略上，因此 $\Psi(\mathbf{1}_o)$ 是相应极端策略的状态，DeleteAll 包含保留下限，无预算时 RepairAll 修复每个候选。连通紧集 Δ 的连续像是连通紧集，Δ 六条棱的像是连接各锚点的曲线。

有界差分: 设 o 与 o' 只在第 i 个坐标上不同。两次执行的差别在于单元格 e_i 的取值，至多两行的保留状态，即 e_i 所在行，和保留下限只在其中一次执行中允许的第一次后续删除所在行，以及之后至多 ξ 个候选的 VE 替换估计值，VEC 从当前表计算这些估计值。被 VE 替换的候选无论写入什么值都算已解决，因此这些估计值不改变已解决集合。令 U 与 U' 为保留行中的不动候选数，n_r,n_r' 为保留行数。则 $|U-U'|\le 2\bar{k}$，$|n_r-n_r'|\le 2$，$n_r,n_r'\ge\rho n$ 且 $U\le K$，因此

$$|A(o)-A(o')|\le\frac{|U-U'|}{n_r' d}+\frac{U}{d}\Big|\frac{1}{n_r}-\frac{1}{n_r'}\Big|\le\frac{2\bar{k}}{\rho n d}+\frac{2K}{\rho^2 n^2 d}$$
$$\le\frac{1}{\rho n}\Big(\frac{2\bar{k}}{d}+\frac{2K}{\rho n d}\Big)=:c_A.$$

对多样性，$V=\frac{1}{d}\sum_c\frac{n_r}{n}\frac{\sigma_c^2(D')}{\sigma_c^2(D)}$。在取值位于 $[-b,b]$、规模 $m\ge\rho n$ 的样本中改变一个值或删除一行，样本方差至多变化 $\frac{8b^2}{m-1}$，因子 $\frac{n_r}{n}$ 至多变化 $\frac{1}{n}$。改变的单元格、至多 ξ 个重新估计的单元格与至多两行合计至多带来 $3+\xi$ 次这样的变化。因此

$$|V(o)-V(o')|\le\frac{3+\xi}{s_{\min}^2}\Big(\frac{8b^2}{\rho n-1}+\frac{2b^2}{n}\Big)=:c_V.$$

集中性: 在 π_p 下 o 的各坐标相互独立。对 A 与 V 分别应用 McDiarmid 不等式，对 $t>0$ 有 $\Pr[|A(\pi_p)-\Psi_A(p)|\ge t]\le 2\exp\big(-\frac{2t^2}{Kc_A^2}\big)$，对 c_V 同理。取 $t=\max(c_A,c_V)\sqrt{\frac{K}{2}\log\frac{4}{\delta}}$，并对两个坐标取并集界，得到 $\|\Phi(\pi_p)-\Psi(p)\|_\infty\le t$ 的概率至少为 $1-\delta$。由于 $\rho n\ge 2$ 给出 $\rho n-1\ge\frac{\rho n}{2}$，且每行至多含 $\bar{k}$ 个候选，$n\ge\frac{K}{\bar{k}}$，因此 $\max(c_A,c_V)\le\frac{C'}{n}\le\frac{C'\bar{k}}{K}$，其中 C' 依赖于 $\bar{k},\xi,\rho,b,s_{\min}^2$ 与 $\frac{K}{nd}$，并且 $t\le C\sqrt{\frac{1}{K}\log\frac{4}{\delta}}$，$C=\frac{C'\bar{k}}{\sqrt{2}}$。概率为正的事件必有实现，该实现就是定理所述的确定性策略。证明的任何一步都不使用 A 或 V 在单元格上的可加性，删除造成的行级耦合只通过 $\bar{k}$ 与 ρ 进入。

方向: 在 $(\Delta A,\Delta R_{sam},\Delta R_{var},cost)$ 中，不动给出 $(0,0,0,0)$。op_{gt} 与 op_{ve} 不删除行而解决一个候选，这把 A 提高 $\frac{1}{n_r d}$，不改变 R_{sam}，并使一列方差变化 $O(\frac{1}{n})$，其中只有 op_{gt} 消耗预算。被执行的删除移除候选所在行，使每一列的样本保留因子 $R_{sam}=\frac{n_r}{n}$ 乘以 $\frac{n_r-1}{n_r}$，因此它是唯一 $\Delta R_{sam}\neq 0$ 的操作，并且当被删行所含未解决候选多于 $\frac{U}{n_r}$ 时，它恰好提高 $A=1-\frac{U}{n_r d}$。op_{gt} 按定义写入真实值，op_{ve} 只在 VEC 能恢复时写入真实值。□

B 定理 2 的证明

隶属性: 证书是一个策略 $\pi:E\to\mathcal{O}$。其成本是 K 项之和，计算 $Perf(M;D_\pi)$ 需要一次施加 π、一次训练与一次评测。实例给出每个候选的 GT 值，算法 A1 的每一级对每条规则扫描一次表，DC 级返回被违反子句中的常数，因此施加 π 是多项式时间，训练与评测按对 M 的条件也是多项式时间，随机学习器使用固定种子。

逐格成本: 逐格成本包含下面的单位成本情形，因此逐格成本下的困难性由它得出。

单位成本: 设 $G=(\mathcal{U},\mathcal{F})$ 与 k_G 是一个 Vertex Cover 实例，$m=|\mathcal{F}|$。实例直接给出编码后的特征，分类器与 VE 替换都在这些特征上使用欧氏距离。在 $\mathbb{R}^{N_0}$ 中构造，每个顶点一个坐标，每条边一个坐标，辅助行另用私有坐标，并取 $h=8$ 与 $L=7$，使 $r=\sqrt{\frac{h^2}{2}+L^2}=9$，再取有理数 $\tau\in(0,1)$、$\eta=\frac{\tau^2}{8r}$ 与有理数 $\epsilon<\eta$，使每个坐标都是有理数。规则库为空。所有标签都是二值的，每个验证点的真实标签为 1。

- 顶点行。对 $u\in\mathcal{U}$，构造训练行 $t_u=h\mathbf{1}_u$，$\mathbf{1}_u$ 为单位向量，真实标签为 1，观测标签为 0。它的标签单元格是该行唯一的候选，成本为 1。
- 验证点。对 $(u,w)\in\mathcal{F}$，构造点 $v_{uw}=\frac{h}{2}(\mathbf{1}_u+\mathbf{1}_w)+Lf_{uw}$，其中 f_{uw} 是私有单位向量。于是 $\|v_{uw}-t_u\|=\|v_{uw}-t_w\|=r$，而其他每个顶点行与 v_{uw} 的距离为 $\sqrt{\frac{3h^2}{2}+L^2}>r$，为其他边构造的锚点行或阻挡行与 v_{uw} 的距离至少为 $\sqrt{2}L-\epsilon>r+3\eta$。
- 锚点行与阻挡行。干净行 $a_{uw}=v_{uw}+\epsilon g_{uw}$，$g_{uw}$ 为私有单位向量，标签为 1，以及位于私有轴上、与 v_{uw}

距离为 $r+\eta$ 与 $r+2\eta$ 的干净行 β_{uw}, β'_{uw}，标签为 0。这些行不是候选，任何操作都不能触及它们。

- 诱饵行。在每个 t_u 周围沿私有轴距离 τ 处放置五条标签为 0 的干净行。它们与 v_{uw} 的距离为 $\sqrt{r^2+\tau^2} > r+2\eta$，其余每条训练行与 t_u 的距离至少为 $r > \tau$，因此这五条诱饵行是 t_u 最近的五条保留行。

v_{uw} 最近的三条训练行是 a_{uw}, t_u, t_w，在观测标签 $(1,0,0)$ 下该点被误分类，当且仅当 t_u 或 t_w 被修复时它变为正确。删除 t_u 会把 β_{uw} 带入邻域，得到 $(1,\ell_w,0)$，只有在 t_w 被修复时才正确，因此删除从不起作用，删除两个端点得到 $(1,0,0)$。对标签单元格的 VE 替换写入最近五条保留行的多数标签（第 E 节），这五条行正是诱饵行，于是 op_{ve} 写入标签 0，等同于 op_{noop}。因此在 k_G 次单位成本修复下被正确分类的验证点数最大值，等于 k_G 个顶点所能覆盖的边数最大值，精度 1（即 $\theta = 1$）在预算 $B = k_G$ 下可达，当且仅当 G 存在大小至多为 k_G 的顶点覆盖。该实例有 $|\mathcal{U}|(1+5)+3m$ 条训练行与 m 个验证点，坐标多项式有界，因此归约是多项式的，而 Vertex Cover 是强 NP 完全的。 □

C 质量状态的充分性

命题 1 用纤维振荡 ω 界定在 (A,V) 中规划的损失，而定义 6 的多样性 V 只记录样本量与逐列方差。本节在九个基准数据集上测量 ω，并检验 (A,V) 未编码的三类统计量，即相关结构、类别分布与联合分布的形状，能否在 (A,V) 之外进一步预测下游得分。

设置。每个数据集上抽取 600 个作用于定义 9 四种操作的随机策略。每个策略先抽取操作概率 $p \sim \mathrm{Dirichlet}(1,1,1,1)$，以概率 p_{del} 删除含 oracle 候选的每个训练行，再对保留行中的每个候选按其余三个概率的比例执行 no-op、GT 修复或 VE 替换，VE 的估计值取 ReplaceAll 的值。对每个输出 D' 及其编码 X'，记录 A 与 V（V 只对脏训练划分中有方差的列取平均）、相关距离 $\frac{\|C(X')-C(X_{clean})\|_F}{\|C(X_{dirty})-C(X_{clean})\|_F}$（$C$ 为列相关矩阵）、D' 与干净训练划分的标签分布之间的全变差距离、X' 与 X_{clean} 之间的切片 Wasserstein 距离，以及首选模型的得分（截断到 $[0,1]$，计算方式同表 IV）。`har` 的轮廓系数在保留行的一个固定的 10,000 行子样本上计算。DemandClean 的输出按同样方式测量。我们把随机策略的 (A,V) 范围划为 20×20 的网格，对至少含三个策略的格子 q，取其中策略得分的极差为 $\omega(q)$。

纤维振荡。表 A1 将 ω 与干净数据相对 NoFix 的增益 G 并列。九个数据集上振荡的中位数都小于 G，且至多为 0.073，因此质量状态相同的策略得分通常相近。`nasa`、`soilmoisture` 与 `mercedes` 的尾部更重，单个格子内的最大振荡分别达到 0.535、0.843 与 0.379。在这些数据集上，命题 1 的界 $2\sup_q \omega(q)$ 较松，这正是 10 维状态的局部特征所针对的情形（第 IV-B2 节）。

(A,V) 之外的统计量。在 (A,V) 之外加入相关距离、标签分布距离与切片 Wasserstein 距离，在九个数据集中的六个上使得分的交叉验证 R^2 至多提高 0.03。`soilmoisture` 上提高 0.07，`beers` 与 `har` 上提高 0.18，因此在这些数据集上相关结构与分布形状携带 (A,V) 之外的信息。在 `beers` 与 `har` 上，DemandClean 的输出在相关结构与分布形状两方面都比至少 99% 的随机策略更接近干净训练划分，并且在 `beers` 上保持 NoFix 的标签分布。在 `mercedes` 与 `smartfactory` 上，DemandClean 的输出比多数随机策略更偏离干净数据的相关结构，得分却高于全部随机策略，这是第 V-C1 节所述保留的方差比贴近干净数据更重要的情形。

DemandClean 所处的位置。在 `adult` 以外的八个数据集上，DemandClean 的得分高于在标准化 (A,V) 平面中离它最近的 20 个随机策略中的至少 90%，也高于全部随机策略中的至少 91%。这一结果不来自 GT，在读取 GT 单元格数不少于 DemandClean 的随机策略中，DemandClean 在存在这类策略的七个数据集上都高于其中至少 85%。因此在 Q 的一个格子内，DemandClean 的策略落在纤维的高端，即命题 1 未消除的那部分规划误差。在 `adult` 上全部随机策略的得分极差不到 0.01，DemandClean 只使用校准种子集，其得分比最好的随机策略低 0.005。

TABLE A1
九个基准数据集上质量状态的充分性。G：干净数据相对 NoFix 的增益。ω_{50}、ω_{90}：网格各格子纤维振荡的中位数与 90 分位数。R^2_{AV}、R^2_{+}：以 (A,V)，以及以 (A,V) 加三个统计量预测得分的 10 近邻回归器的五折交叉验证 R^2。近邻、全部：在 (A,V) 中离 DemandClean 最近的 20 个随机策略，以及全部 600 个随机策略中，得分低于 DemandClean 的比例（%）。

数据集	G	ω_{50}	ω_{90}	R^2_{AV}	R^2_{+}	近邻	全部
`adult`	.005	.003	.004	.10	.09	15	26
`beers`	.040	.029	.044	.39	.57	90	99
`breast_c.`	.029	.014	.021	.33	.34	95	98
`smartf.`	.005	.004	.007	.77	.80	100	100
`bike`	.141	.011	.020	.96	.96	100	100
`mercedes`	.034	.012	.023	.43	.43	100	100
`nasa`	.532	.000	.177	.81	.83	90	100
`soilm.`	.102	.045	.082	.47	.55	100	91
`har`	.408	.073	.151	.78	.97	100	100

D 方法细节

检测示例。图 2（步骤 1 示例）在 Beer 子集上演示了这四个阶段。阶段 1 中，IBU 单元格被标为缺失（`null`），但此阶段信息不足以修复。阶段 2 中，ABV = 178.7 被域规则 `ABV<100` 标记，并被临时校正为该列最常见的 ABV 值 45。ABV 列稳定之后，阶段 3 可靠地求值 CFD `ABV<50` ⇒ `City=Gary`，并把域内的 `City=Chicago` 标为语义错误。

推理模式。训练好的智能体以纯利用模式（$\varepsilon = 0$）运行，经第 IV-B2 节的两阶段决策逐个处理检出错误，如图 2 的两阶段推理面板所示。第一阶段（智能体决策）为每个错误单元格给出一个操作，零成本动作（op_{noop}、op_{del}、op_{ve}）立即执行。GT 修复（op_{gt}）以两种模式之一实现（算法 3）。在线模式下，标签在决策时即可获得，每个 GT 修复当场写入真实值，后续状态因此是精确的。离线模式下，每个被指派 GT 修复的单元格记入修复计划 $P = \{(row_i, col_i, v_i)\}_{i=1}^{|P|}$，并临时写入其 VEC 估计 v_i 以维持状态连续性。计划的生成因此不触碰任何 GT（图 2 中 ①），标注需求也由此从 $|E|$ 降到 $|P|$（$|P| \ll |E|$）。第二阶段（动作执行）随后写入用户为 P 的任意子集提供的 GT（②），用户因此可以分批、延后完成对 P 的标注。

同一格子中的策略。同一格子中的两个策略在两种情况下性能不同，一是它们用不同操作解决候选，因为交换 GT 修复与 VE 替换不改变 A（定理 3(i)），而只有 GT 修复总能恢复真实值，二是它们删除数目相同但位置不同的行，这会改变模型看到的训练样本。因此仅凭质量状态无法区分同一格子中的策略，10 维状态（第 IV-B2 节）补充了这部分信息。其中保留行比例（s_6）、当前列的方差比（s_7）与剩余错误比例（s_9）刻画决策进度与局部质量状况，局部特征与预算感知进一步区分格子内的策略，使智能体能在逐格序贯决策下向最优格子移动。这些特征的计算方式与数据集的领域、特征数或错误模式无关，因此同一个智能体可以不经修改地在另一个数据集上运行（第 V-D 节）。由于状态逐格计算，智能体在候选不同的数据集上选择不同的操作，这与第 V-C3 节随数据集而变的归因结果一致。例如，当剩余预算比例（s_8）低于

Algorithm A1 特征单元格 (i, j) 的值估计链

Require: 当前表 X，已删除行 R_{del}，列均值 μ，规则库 $\mathcal{R}$，原始脏字符串 $\tilde{x}_{ij}$
Ensure: 估计值 $\hat{v}$
// L1 FD
for $\mathcal{R}$ 中每条满足 $X_{i,L}$ 无缺失的 FD $L \Rightarrow j$ **do**
  $S \leftarrow \{i' \neq i,\ i' \notin R_{del} : X_{i',L} = X_{i,L},\ X_{i'j}$ 非缺失$\}$
  if $|S| \geq 2$ **return** clip_j（j 为类别列或 $X_{S,j}$ 至多有 5 个不同取值时取 $X_{S,j}$ 的众数，否则取 $X_{S,j}$ 的均值）
end for
// L2 DC
for 每条标记列 j 且 j 以外的子句在行 i 上全部成立的 DC **do**
  if 它在 j 上的子句为 $x_j \neq a$、$x_j > a$、$x_j \geq a$、$x_j < a$ 或 $x_j \leq a$ **return** $\mathrm{clip}_j(a)$
  if 该子句为 $|x_j - x_l| > t$ **return** $\mathrm{clip}_j(X_{il})$
end for
// L3 CFD
if 某条以 j 为目标的 CFD 声明了基线值 b **return** $\mathrm{clip}_j(b)$
// L4 数值提取
if j 为数值列，X_{ij} 缺失，且 $\tilde{x}_{ij}$ 含有匹配 `-?\d+(\.\d+)?` 的子串 **return** clip_j（第一个匹配）
// L5 编辑距离
$c^\star \leftarrow \arg\max_c \mathrm{sim}(\tilde{x}_{ij}, c)$，$c$ 取遍列 j 的已知类别
if j 为类别列，$\mathrm{sim}(\tilde{x}_{ij}, c^\star) \geq 0.6$，$c^\star \neq \tilde{x}_{ij}$，且 X_{ij} 不是列 j 的众数 **return** $c^\star$
// L6 近邻
$N \leftarrow$ 满足 $i' \notin R_{del} \cup \{i\}$ 且 $X_{i'j}$ 非缺失、在其余列上与行 i 欧氏距离最近的 5 行，缺失项以 μ 填充
$w_{i'} \propto \frac{1}{dist(i,i')+10^{-8}}$
if $N \neq \emptyset$ **return** clip_j（j 为类别列时取 $X_{N,j}$ 的加权投票，否则取 $\sum_{i' \in N} w_{i'} X_{i'j}$）
// L7 列统计量
return clip_j（j 为类别列时取列 j 的众数，否则取 μ_j）

剩余错误比例（s_9）时，训练好的智能体选择零成本 VE 替换的次数多于 GT 修复。

E 实现细节

数据划分。每个数据集按 60%、20% 与 20% 的比例划分为训练、验证与测试行。各基线方法以及报告的每一次 DemandClean 运行都使用这一划分。类别特征做整数编码，数值特征做标准化，编码器只在脏训练行上拟合。

候选下游模型。第 IV-B1 节基于容忍度的选择在分类任务上从随机森林（RF）、逻辑回归（LR）、SVM、k 近邻（KNN）、决策树（DT）与梯度提升（GB）中选取，在回归任务上从 RF、线性回归（LR）、Ridge、Lasso、KNN 与 GB 中选取，在聚类任务上从 KMeans、层次聚类与谱聚类中选取。

值估计链。算法 A1 按既定优先级依次检查各估计级，采用首个返回取值的一级。每一级的返回值都经过投影 clip_j，它把整数列或实数列 j 的取值投影到声明的 `[DOMAIN]` 范围内，其他列保持不变。对标签单元格执行 VE 替换时，分类任务写入 5 个最近保留行的多数标签，回归任务写入这 5 行按距离倒数加权的均值。

状态。表 A2 给出表中第 i 行第 j 列候选的十个坐标，该表有 n 行与 d 个特征列。E 是该 episode 的候选集，k 是当前候选在处理顺序中的位置，n_r 是保留行数，B 是以成本单位计的 GT 预算，C 是已花费的成本。s_1 与 s_2 由奖励使用的代理模型给出。已删除行中的候选在 s_0 至 s_7 上取 0。

奖励。代理得分 sc 在分类任务上是准确率，在回归任务上是 $\frac{1}{1+\ln(1+\mathrm{MSE})}$，在聚类任务上是 $\frac{1+sil}{2}$，它在验证划分上计算。sc_0 是在 episode 表中不含候选的行上拟合的代理模型得分，sc_k 是第 k 步在保留行上重新拟合代理模型后的得分。记

TABLE A2
候选单元格 (i, j) 的 10 维状态。标签单元格（j 为标签列）取最后一列的值。

	特征单元格	标签单元格
s_0	类型 $t \in \{0,1,2,3\}$（缺失、语义、语法、标签）取 $\frac{t}{3}$	1
s_1	列 j 在代理模型中的重要性，归一化为和为一（RF 的不纯度重要性、Ridge 系数绝对值、KMeans 的质心平均偏离）	1（KMeans 取最大值）
s_2	分类取 $2\max_c \hat{p}(c \mid x_i) - 1$，回归取 $\frac{\lvert\hat{y}_i - \bar{y}\rvert}{2\sigma_y}$，聚类取 $\delta_2 - \delta_1$，δ_1, δ_2 为到最近两个质心的距离，三者都截断到 $[0,1]$	同左
s_3	$\frac{i}{n-1}$	$\frac{i}{n-1}$
s_4	$\frac{j}{d-1}$	1
s_5	$\frac{\lvert E_j\rvert}{\lvert E\rvert}$，列 j 中候选所占比例	0.5
s_6	$R_{sam} = \frac{n_r}{n}$	$\frac{n_r}{n}$
s_7	$R_{var} = \min\big(1.5, \frac{\sigma_j^2(\text{保留行})}{\sigma_j^2(\text{episode 开始时})}\big)$	1
s_8	$\frac{\max(0, B-C)}{\max(B,1)}$	同左
s_9	$\frac{\lvert E\rvert - k}{\lvert E\rvert}$	同左

$\pi_k = \frac{(1-s_2)+s_1+s_5}{3}$ 与 $g_k = \frac{1}{1+e^{-6(s_8-s_9)}}$，动作 a_k 的塑形项为

$$r_k^{sh} = \kappa\, w(t) \cdot \begin{cases} -0.03 & a_k = op_{noop}, \\ (0.02 + 0.04\,\pi_k)\, g_k & a_k = op_{gt}, \\ \delta_{del} & a_k = op_{del}, \\ 0.03 & a_k = op_{ve}, \end{cases}$$

其中 $\kappa = \min\big(10, \max(0.1, \frac{m}{0.03})\big)$。$m$ 是 $|sc_k - sc_0|$ 在该 episode 各评估步上的指数平均，初值取第一个评估步的值，之后按 $m \leftarrow 0.3\,|sc_k - sc_0| + 0.7\,m$ 更新，第一个评估步之前 $\kappa = 1$。塑形权重在 T 个 episode 的前一半取 $w(t) = 1$，之后取 $w(t) = \max\big(0.1,\ 1 - 0.9\,\frac{2t-T}{T}\big)$。第一步以及每隔 N 步的一步是评估步，奖励为 $sc_k - sc_0 - \lambda c_k \mathbf{1}[a_k = op_{gt}] + r_k^{sh}$，其余步只得到 r_k^{sh}，且

$$N = \max\Big(5,\ \min\big(\max(1, \lfloor\tfrac{|E|\,n}{5\cdot 10^5}\rfloor),\ \max(1, \lfloor\tfrac{|E|}{20}\rfloor),\ 200\big)\Big).$$

episode 的最后一步以 $R_{final} = 5(sc_T - sc_0) + \beta\big(1 - \frac{|R_{del}|}{n}\big) - \lambda C$ 代替该步奖励。预算为 $B = \lfloor r_{\max}|E|\rfloor$ 个成本单位，满足 $C + c_k > B$ 的 GT 修复请求按 VE 替换执行，已删除 $\lfloor 0.8\,n\rfloor$ 行之后的删除按不动执行。所有报告的运行都使用单位成本 $c_k = 1$。各数据集所用的常数参数见公开代码仓库。

网络训练。每个阶段都是一个 Dueling 网络，含两层各 128 个 ReLU 单元的隐藏层，输出为 $Q(s,a) = V(s) + A(s,a) - \frac{1}{|\mathcal{A}|}\sum_{a'} A(s,a')$，用 Double DQN 目标、平方损失与学习率 $5 \cdot 10^{-4}$ 的 Adam 训练，折扣因子为 0.95，每个阶段的经验回放容量为 10^4 条转移。一个 episode 是对 D_{base} 的一个被污染副本中打乱顺序的候选的一次遍历。每个 episode 结束后两个阶段各在 64 条转移的小批量上更新一次，探索率从 1.0 起每次乘以 0.995，下限为 0.1，每隔五个 episode 目标网络按 $\theta^- \leftarrow 0.1\,\theta + 0.9\,\theta^-$ 更新。阶段 1 存储每一条转移，阶段 2 只存储由 GT 修复或 VE 替换就地解决单元格的转移。训练保留最终代理得分最高的 episode 的权重，得分相同时取 GT 修复较少的 episode，并在结束时恢复这组权重。D_{base} 取两张零成本表中按 5 折交叉验证代理得分乘以 $\sqrt{\frac{n_{kept}}{n}}$ 较高的一张。第一张删除含已检测候选的行，同时至少保留 $\max(10, \lfloor 0.2n\rfloor)$ 行，并用值估计链填补剩余缺失单元

格。第二张对每个已检测单元格执行值估计链，对每个已检测标签执行 5 近邻标签估计。

对抗错误注入。对每种错误类型，检测结果确定一个比例 $\hat{r}$。缺失、语义与语法错误的 $\hat{r}$ 是该类型的检测单元格数除以 nd，标签错误的 $\hat{r}$ 是检测数除以 n。每个 episode 从 $[\ell, u]$ 中均匀抽取注入比例，其中 $\ell = \max(0.8\hat{r}, 0.005)$，$u = \max(1.2\hat{r}, \ell + 0.005)$。没有检测结果的类型取 $[0, 0]$。没有检测到语义错误而规则库含 FD、CFD 或 DC 时，语义错误取 $[0.005, 0.02]$。D_{base} 有 n_b 行时，注入器对每种类型在未用过的随机位置尝试注入 $\lfloor n_b r \rfloor$ 个单元格，每个单元格注入前的值就是它的伪真值。缺失错误把单元格置空。规则库含 CFD 或 DC 时，语义配额的一半用于 CFD 违反，它把目标值越过 CFD 阈值移动 $U(0, 2)$ 个原始单位，同时保持在声明的值域内，另一半用于使某条 DC 的全部子句成立的 DC 违反。这两类放不下的配额用于 FD 违反，它把另一个左部分组的右部值复制到 $g \geq 3$ 行分组中至多 $\lfloor \frac{g-1}{2} \rfloor$ 行，使原值保持多数。没有规则能放置的单元格改写为本列的另一个取值。规则库声明值域时，语法错误把 30% 的配额用于值域违反，把数值放到值域外 1 至 5 个原始单位处，或把枚举值放到最大值之后 1 至 3 步。其余配额按 40%、30% 与 30% 分给三种错误，即列均值加减 $U(3, 5)$ 倍标准差的高斯偏移、越过 1% 或 99% 分位数且超出量为该分位区间 $U(0.3, 1.0)$ 倍的值，以及把值乘以 10、取整后乘以 11（如 $3 \to 33$）或移到零的另一侧的量级错误。这三种错误只作用于 FD 左部以外的列，在类别列上改为替换成另一个合法类别。只有检测器报告了标签错误时才注入标签错误，分类任务按检测到的翻转模式翻转，回归任务加检测到的离散程度的高斯噪声，注入行不含已注入的特征错误。

检测。集成检测器的校准种子集在九个基准数据集上由 20 至 80 条标注记录组成，这些记录的全部 120 至 7,540 个单元格都计入 GT%。`flights`、`soccer` 与 `hospital` 上的运行关闭了该检测器，检测阶段不读取标注单元格。

基线。各基线方法在上述划分的训练行上运行。DAGPipe 还将这些训练行上学到的计算图应用于测试输入，评估时固定图结构与算子参数。统一评估器在每份输出上训练表 III 的下游模型，并在测试行上打分，聚类任务除外，其轮廓系数在清洗后的训练行上计算。GT 统计方法在训练行上读取的干净表单元格数。我们的代码仓库（https://github.com/qzkinhit/DemandClean）给出每个基线方法的实现、它对 GT 与验证划分的使用，以及回归、聚类和天然脏数据集所需的适配。以评估器的编码输出清洗后矩阵的方法（DiffPrep、DIANA 与 Learn2Clean）在该矩阵上打分，DAGPipe 在其计算图产生的特征上打分。

规则库。DemandClean 读取 `data/<ds>/rules.txt` 中的规则库 $\mathcal{R}$，其 `[DOMAIN]`、`[FD]`、`[CFD]`、`[DC]` 与 `[REGEX]` 各节供检测器、值估计链与注入器使用。HoloClean、UniClean 与 Horizon 读取 Clean4ML 基准发布的规则文件 `benchmark/Data/<ds>/rules.txt`，其中 `[HOLOCLEAN_DC]`、`[UNICLEAN]` 与 `[HORIZON_FD]` 三节用各方法自己的语言写出依赖。在三个天然脏数据集上，它们读取随 UniClean 发布的否定约束、清洗算子与依赖。表 A3 统计两份文件中的规则数。两份文件在 `adult`、`beers`、`har` 与 `nasa` 上含相同的函数依赖。在 `bike` 上，$\mathcal{R}$ 用六条单元组否定约束表达季节与月份的关系，另用三条把 holiday、weekday 与 workingday 关联起来，基线方法读取 `mnth` $\Rightarrow$ `season`。

$\mathcal{R}$ 的 `[DOMAIN]` 节对每个受约束的列列出该列的取值范围或取值集合。`soilmoisture` 的数值边界与 `bike` 日期列的边界经过取整，`beers` 的 `ounces` 集合列出容器规格。在天然脏数据集上，值域约束标签列，`hospital` 上另约束两个类别列。表 A6 的实验还用只由脏表导出的规则库部署 DemandClean，其数值值域取脏列的 0.5% 与 99.5% 分位数，类别值域取出现至少两次的取值，FD 从脏表中挖掘。

TABLE A3
各数据集的规则数。左侧为 DemandClean 规则库 $\mathcal{R}$ 按节的规则数。右侧为 Clean4ML 规则文件中 HoloClean 的否定约束、UniClean 的清洗算子与 Horizon 的 FD 数。

	DemandClean 的 $\mathcal{R}$					Clean4ML 文件		
数据集	值域	FD	CFD	DC	正则	HC	UC	Hor
`adult`	6	1	24	0	0	1	6	1
`beers`	4	4	0	0	0	4	5	4
`bike`	16	0	0	9	0	1	8	1
`breast_c.`	10	0	22	8	3	1	9	1
`har`	4	1	0	0	0	1	6	1
`mercedes`	9	0	0	0	0	2	10	2
`nasa`	6	3	0	0	0	3	9	3
`smartf.`	19	0	0	0	0	0	5	12
`soilmoist.`	127	0	0	124	0	0	10	1

TABLE A4
不经重训跨同类型数据集部署的智能体的 Tol_{rel}。行为智能体的训练数据集，列为目标数据集，粗体对角线为在目标数据集上训练的智能体。

分类 Source \ Target	`adult`	`beers`	`breast_c.`	`smartf.`
`adult`	**.31**	−.37	.25	−.45
`beers`	−.90	**1.00**	1.00	1.14
`breast_c.`	−.52	1.00	**1.00**	.73
`smartf.`	−.90	.53	1.00	**.82**
回归 Source \ Target	`bike`	`mercedes`	`nasa`	`soilm.`
`bike`	**1.00**	1.35	.08	.78
`mercedes`	.72	**.53**	.07	.67
`nasa`	1.00	1.26	**1.00**	.81
`soilm.`	.94	.88	.00	**.79**

F 附加结果

不重训的迁移。在一个数据集上训练的智能体不经重训，使用目标数据集的检测器，部署到同一任务类型的其他每个数据集上，表 A4 报告它在目标数据集上的 Tol_{rel}。表中每一格，对角线在内，都按同一方式重新部署一个训练好的智能体，因此每一列都比较迁移来的智能体与目标数据集自身的智能体。在 `beers`、`breast_cancer`、`smartfactory`、`bike`、`mercedes` 与 `soilmoisture` 上，至少有一个迁移来的智能体持平或超过在目标数据集上训练的智能体，在 `mercedes` 上每个迁移来的智能体都超过了它。在 `adult` 与 `nasa` 上，在目标数据集上训练的智能体领先于所有迁移来的智能体。

消耗 GT 的基线。表 A5 让 ActiveClean、Comet、Lopster 与 RetClean 在两种 GT 水平下运行，并与 DemandClean 部署的运行对比。在 DemandClean 的预算下，ActiveClean 清洗的记录数与 Lopster 学习的干净记录数都以 DemandClean 的 GT 单元格所能覆盖的条数为准，Comet 可查询的 GT 值数等于 DemandClean 的种子记录数与 GT 修复数之和。RetClean 从干净值构成的数据湖中检索修复依据，较低水平下湖中存放全部单元格的 1.65% 与 DemandClean 在该数据集上读取的 GT 单元格数两者中的较大者，因此湖中的 GT 从不少于 DemandClean。在默认水平下，ActiveClean 与 Comet 使用各自的默认预算，Lopster 学习训练行中一半行的干净值，RetClean 的湖中存放随机固定的一半训练行的干净值。两种水平下 Comet 的 GT 都包含其错误掩码。RetClean 在 `adult` 上找不到目标单元格，不读取 GT。`flights`、`soccer` 与 `hospital` 不在表中，

TABLE A5
四个从 GT 学习的基线在不低于 DemandClean 的 GT 预算与各自默认 GT 用量下的下游得分与 GT%，打分方式同表 IV。

	DemandClean		ActiveClean				Comet				Lopster				RetClean			
			DemandClean 预算		默认		DemandClean 预算		默认		DemandClean 预算		默认		≥ DemandClean 预算		默认	
	Score	GT%	Score	GT%	Score	GT%	Score	GT%	Score	GT%	Score	GT%	Score	GT%	Score	GT%	Score	GT%
adult	.854	0.1	.851	0.1	.854	1.1	.852	56.0	.852	56.0	.805	0.1	.764	30.0	.852	0	.852	0
beers	.361	1.8	.326	1.7	.340	20.7	.340	48.0	.344	48.0	.111	1.7	.173	30.0	.303	1.8	.353	27.0
breast_c.	.993	2.9	.957	2.9	.986	29.9	.957	53.9	.979	53.9	.964	2.9	.943	29.9	.986	3.0	.986	29.9
smartf.	.969	0.2	.962	0.2	.963	2.1	.962	56.8	.966	56.8	.702	0.2	.642	30.0	.965	1.6	.962	30.0
bike	1.00	1.0	.895	1.0	.943	2.9	1.00	56.2	1.00	56.2	.955	1.0	.984	30.0	.983	1.6	.982	30.0
mercedes	.428	2.0	.394	2.0	.368	11.9	.360	59.8	.363	59.8	.401	2.0	.408	30.0	.360	2.0	.358	30.0
nasa	.533	6.5	−.001	6.5	.000	29.9	.532	50.0	.390	50.0	.164	6.5	.417	29.9	.509	6.5	.515	29.9
soilm.	.940	2.9	.872	2.9	.937	29.9	.857	59.5	.857	59.5	.868	2.9	.860	29.9	.904	2.9	.913	29.9
har	.899	0.1	.499	0.1	.506	0.7	.504	3.2	.520	3.2	.747	0.1	.866	30.0	.799	1.6	.802	30.0

TABLE A6
退化规则库下的检测 F1 与 Tol_{rel}，九个数据集上的平均。每个删除比例与虚假约束规则库取三次随机抽取的平均。

规则库	检测 F1	Tol_{rel}
完整	.84	.83
删除 25% / 50% / 75%	.84 / .82 / .80	.78 / .70 / .64
加入错误约束	.46	.31
由脏表推导	.72	.48

因为 DemandClean 在其上至多读取四个 GT 单元格，在这一预算下 ActiveClean、Comet 与 Lopster 的得分都与 NoFix 完全相同。在这三个数据集上，RetClean 在较低水平下得分为.983、1.00 与.995，在默认水平下为.998、1.00 与.995，DemandClean 均为 1.00。

对规则库的鲁棒性。每个训练好的智能体都按同一方式重新部署，分别配合完整规则库、删去四分之一到四分之三约束的规则库、加入虚假依赖与收窄值域的规则库，以及只由脏表推导的规则库（表 A6）。删去约束时结果平缓下降。加入虚假约束的危害最大。漏检的单元格不会进入候选集，因此检测召回率限定了可达增益。

各数据集的 GT 用量。表 A8 列出表 IV 中 GT% 列与图 4 中 GT 面板背后的逐数据集 GT 用量，按第 V-B 节指标 ⑤ 计数。三种 oracle 检测参照策略读取全部干净训练单元格，Comet 通过错误掩码读取它所处理的训练行的干净值，表中未列出的基线不读取任何干净值。

GT 分配。DemandClean 把 GT 花在数据侧仍有提升空间的地方。在 adult 上，表 IV 中四种参照策略相差不到 0.005，所选模型的先验容忍度在任何标注之前就接近零，DemandClean 因此除校准种子集外不花费任何 GT。在 smartfactory 上，修复全部错误同样只比 NoFix 高不到 0.005，DemandClean 只用 0.2% 的单元格作为 GT 就超过了干净数据。在 hospital 上，脏数据已经达到最高分，RepairAll 与多数基线也是如此，DemandClean 不用任何 GT 也达到最高分。在 soilmoisture 上，DemandClean 只以校准种子集作为 GT，就恢复了干净数据的大部分增益。

天然脏数据集。表 A10 列出表 IV 中的四个天然脏数据集，并给出 FM 基线的输入 token 数与费用。在全部十二个数据集上，FM 基线按标价共花费 207.30 美元，其中 mercedes 上为 98.27 美元。

运行时间。表 A7 列出每种参照策略、检测、模型选择，以及 DemandClean 与四个基线的挂钟运行时间。运行时间在一台配备 16 GB 统一内存的 Apple M2 SoC 上测得。训练占主导，从 37 秒（breast_cancer）到 6.7 小时（har）不等。每个 episode 做 $O(|E|)$ 次两个浅层 MLP 上的前向计算，加上大致恒定次数的代理重拟合，不需要 GPU。har（70K 行）耗时 6.7 小时，每个评估步奖励都在它的全部行上重拟合代理，而 mercedes（4K 行，181K 个错误单元格）在 1.6 小时内完成，Ridge 重拟合只涉及 2.5K 行。推理从不到 1 秒（nasa）到 200 秒（bike）不等，在九个数据集中的八个上短于 Raha+Baran、HoloClean 与 UniClean 的端到端时间。训练对每个数据集只运行一次，此后在该数据集上的每次清洗只花费这一推理时间。

变体 1：DemandClean-ND（oracle 检测）。将第 IV-A 节的检测器替换为 oracle 错误单元格后，八个数据集上的 Tol_{rel} 全部下降，在 soilmoisture 上低于 NoFix。mercedes 与 smartfactory 上检测器本已接近准确（$P_d \geq 0.98$，$R_d = 1.00$），损失不可能来自检测质量。变化的是智能体看到的带类型错误集，因为 oracle 路径绕过了算法 1 的分阶段定型和临时 VEC 修复，状态中的错误类型（s_0）与列错误率（s_5）随之偏移，即便标记的单元格完全相同。因此完整 DemandClean 比该变体更强，也更稳定。它的均值 1.22 超过干净数据，该变体为 0.61，它在八个数据集上的四分位距为 0.32，不到该变体 0.76 的一半。

变体 2：DemandClean-NGT（零预算，$B = 0$）。零预算智能体能保住的正是发现 3 中 VE 替换承担收益的数据集，在需要精确修复或删除的地方则跌到 NoFix 底线或以下。在 nasa 上，零成本的 ReplaceAll 参照已达到 0.98 的 Tol_{rel}，单独训练的零预算智能体只有 0.25，在 mercedes 上，真值承担了 0.98 的增益，它跌到 NoFix 以下。下降来自推理期缺少标签，也来自阶段 2 中区分 GT 修复与 VE 替换的成本信号在训练时就被移除。

变体 3：DemandClean-Arch（Dueling 网络整体替换为普通 DQN）。该变体把 DemandClean 的整个网络配置（Dueling 头、两层各 128 单元的隐层与软目标更新）替换为 64-64-32 的普通 DQN，其目标网络每隔五个 episode 复制一次在线网络的权重。这一替换使图 5 中的平均 Tol_{rel} 从 1.22 降到 0.98，损失最大的是 beers、mercedes 与 smartfactory。

变体 4：DemandClean-Stage（两阶段扁平为单阶段）。扁平化后 GT 修复与三个零成本操作在同一个 argmax 中比较。平均值降到 0.93，损失最大的是 beers（0.47，完整方法为 1.16）、nasa 与 smartfactory。

网络配置（变体 3）。Dueling 网络的输出为 $Q(s,a) = V(s) + A(s,a) - \frac{1}{|\mathcal{A}|}\sum_{a'} A(s,a')$。中心化后的优势在各动作上求和为零，因此一个阶段存储的每条转移不论记录哪个动作都会更新共享的 $V(s)$，在各动作回报接近相等的候选上，估计值主要由 $V(s)$ 决定。第 IV-B2 节所述的大量低影响候选就处于这种情形。去掉 Dueling 头的变体是 64-64-32 的 ReLU 网络，每个动作一个输出，每个输出只由记录该动作的转移拟合，目标网络每隔五个 episode 复制一次在线网络的权重。

两阶段层次（变体 4）。在 DemandClean 中，阶段 1 比较不动、删除与进入阶段 2，每条转移都更新它，因此进入阶段 2

TABLE A7

完整性能与挂钟运行时间。DC = DemandClean。左：4 种 oracle 检测下的参照策略（NF = NoFix，Del = DeleteAll，Repl = ReplaceAll，Repa = RepairAll，即干净数据）与 **DC**（自动检测）。**中**：GT%（含校准种子集）、检测器 P_d/R_d、最佳外部基线（粗体表示超过干净数据）、首选与次选模型及其先验容忍度 $\widehat{Tol}$（第 IV-B1 节），以及 **DC** 在次选模型上的得分 P。**右**：校准种子集的 GT 单元格数（Seed）与挂钟秒数（**DC** 为训练与推理两项，训练含检测，基线为端到端）。**粗体**/下划线：参照策略与 **DC** 中的最优与次优。C/R/Cl 标示任务类型。基线缩写同表 I。

	Oracle 检测参照				自动		检测		对比基线		首选模型		次选模型				DC 时间 (s)			基线时间 (s)			
Dataset	NF	Del	Repl	Repa (clean data)	**DC**	**GT%**	P_d	R_d	**Best Ext.**		**Mod.**	$\widehat{Tol}$	**Mod.**	P	$\widehat{Tol}$	**Seed**	**Train**	**Inf**	**R+B**	**HC**	**UC**	**Lop**
adult^{C}	.852	**.856**	**.856**	**.856**	.854	0.1%	.17	.30	AC	.854	RF	0.03	GB	.868	−0.01	900	13,178	59	1,564	46	33	598
beers^{C}	.315	.307	.330	.355	**.361**	1.8%	.98	.99	RetClean	.353	RF	0.09	GB	.307	0.18	200	92	11	1,825	148	59	44
breast_c.^{C}	.957	.971	.986	.986	**.993**	2.9%	.81	.76	AC	.986	LR	0.60	GB	.979	0.00	200	37	4	305	58	25	28
smartf.^{C}	.962	.846	.962	.966	**.969**	0.2%	.98	1.00	Learn2Clean	**.967**	RF	0.19	GB	.797	−0.02	760	1,732	75	54,922	11,119	4,827	392
bike^{R}	.859	.865	.998	**1.00**	**1.00**	1.0%	.97	.99	Comet	1.00	LR	1.00	LASSO	1.00	1.00	640	4,571	200	71,774	2,965	16,700	236
mercedes^{R}	.360	.359	.408	.394	**.428**	2.0%	1.00	1.00	DIANA	**.402**	RF	0.05	RIDGE	.358	0.09	7,540	5,628	136	37,966	9,771	28,667	1,842
nasa^{R}	−.001	.528	.523	.532	**.533**	6.5%	.95	1.00	RetClean	.515	LR	0.53	RF	.680	0.19	120	53	<1	382	26	92	16
soilm.^{R}	.856	.749	.920	**.958**	.940	2.9%	.62	.86	AC	.937	GB	0.60	RF	.934	0.49	2,560	316	26	1,116	552	40	202
har^{Cl}	.500	.908	.702	**.910**	.899	0.1%	.86	.97	RetClean	.802	Agglo.	0.41	KMeans	.900	0.34	320	24,174	164	1,449	2,611	3,224	255

TABLE A8

读取干净训练值的每个方法在各数据集上的 GT%。

方法	adult	beers	breast_c.	smartf.	bike	mercedes	nasa	soilm.	har	flights	soccer	hospital
DeleteAll/ReplaceAll/RepairAll	60.0	60.0	59.9	60.0	60.0	60.0	59.9	59.9	60.0	60.0	60.0	60.0
Raha+Baran	<0.1	0.8	2.9	0.1	0.1	0.5	1.3	2.9	<0.1	0.8	0.2	2.0
Lopster	0.1	1.7	2.9	0.2	1.0	2.0	6.5	2.9	0.1	0	0	0
ActiveClean	1.1	20.7	29.9	2.1	2.9	11.9	29.9	29.9	0.7	21.0	4.7	30.0
Comet	56.0	48.0	53.9	56.8	56.2	59.8	50.0	59.5	3.2	51.4	54.0	56.8
RetClean	0	27.0	29.9	30.0	30.0	30.0	29.9	29.9	30.0	30.0	30.0	30.0
DemandClean	0.1	1.8	2.9	0.2	1.0	2.0	6.5	2.9	0.1	<0.1	0	0

TABLE A9

在不读取干净值的两玩家归因中写入单元格（$\hat{\phi}_{\mathcal{W}}$）与删除行（$\hat{\phi}_{\mathcal{D}}$）的归一化 Shapley 值、表 V 中的删除份额，以及写入单元格数与删除行数。粗体：$\hat{\phi} > 0.05$，灰色：$\hat{\phi} < -0.05$。

	两个玩家		表 V	计数	
数据集	$\hat{\phi}_{\mathcal{W}}$	$\hat{\phi}_{\mathcal{D}}$	$\hat{\phi}_{\mathcal{D}}$	$\lvert\mathcal{W}\rvert$	$\lvert\mathcal{D}\rvert$
adult	−.13	**.87**	.87	2	919
beers	**1.00**	.00	.00	2,690	0
breast_c.	**.60**	**.40**	.37	148	102
smartf.	**.91**	**.09**	.06	1,589	2,735
bike	**.96**	−.04	−.04	24,927	376
mercedes	**1.00**	.00	.00	58,043	0
nasa	**1.00**	.00	.00	459	0
soilm.	**1.00**	.00	.00	19,314	0
har	.01	**.99**	.99	53	20,513
flights	**1.00**	.00	.00	1,353	0
soccer	**1.00**	.00	.00	901	0
hospital	–	–	–	1,074	143

TABLE A10

四个天然脏数据集与 FM 基线 [1]，打分方式同表 IV。DC 为 DemandClean，Repa、NF 与 Del 为表 IV 的 RepairAll、NoFix 与 DeleteAll，Ext. 为最好的外部清洗方法，GT% 为 DemandClean 的 GT 用量，FM 为 Claude Opus 4.8，输入 token 计入其全部轮次，费用为其输入与输出 token 的标价 [2]。粗体：最好的清洗方法。

数据集	DC	Repa	NF	Del	Ext.	GT%	FM	费用
beers	**.361**	.355	.315	.307	.353	1.8	.338	316K / $1.77
flights	**1.00**	1.00	.941	.834	1.00	<0.1	.941	188K / $0.84
soccer	**1.00**	1.00	.902	.856	1.00	0	.902	5.06M / $5.45
hospital	**1.00**	1.00	1.00	.995	1.00	0	1.00	329K / $1.44

的价值汇总了阶段 2 两种结果的回报。阶段 2 只比较 VE 替换与 GT 修复，并从就地解决单元格的转移中学习，因此它对两种操作的比较恰好在二者成本与恢复值不同的单元格上拟合，预算特征 s_8 与 s_9 通过上文奖励中的按预算余量缩放 GT 修复塑形项的系数 g_k 与成本项 λc_k 进入。扁平变体是一个四输出的 Dueling 网络，只有一个经验回放池，因此 GT 修复在同一个 argmax 中与三个零成本操作竞争，四个动作在每个状态上共用一个价值估计。

同一结局在同一任务类型内也会翻转符号。在分类任务中，真实值在 `adult` 上降低分数，在其余有增益的数据集上都提高分数。在回归任务中，真实值在 `mercedes` 与 `nasa` 上几乎承担全部增益，在 `bike` 上却不到一半，那里估计值承担得更多，删除还略微降低了分数。`mercedes` 上相对 NoFix 的增益来自取得真实值的 5.7 万个单元格，而像 RepairAll 那样恢复全部 18.1 万个标记单元格会收缩方差、分数更低（发现 1），策略也就让 68% 的标记单元格保持不变。符号翻转说明任务类型只能给出先验倾向，替代不了对表格内部状态的逐例决策，这正是挑战 1 的经验形态。

不读取干净值的归因。表 A9 重做表 V 的归因，两个玩家只由每次部署运行的输出与脏训练划分确定。写入单元格 $\mathcal{W}$ 是输出值与脏值不同的保留训练单元格，删除行 $\mathcal{D}$ 是输出中不再出现的训练行。区分写回真实值的单元格与写入估计值的单元格需要干净值，因此表 V 的 $\mathcal{T}$ 与 $\mathcal{E}$ 合并为 $\mathcal{W}$。每个联盟都按表 V 的方式打分，在全部十二个数据集上，空联盟给出 NoFix，全联盟给出该运行自身的分数，整个计算不读取任何干净训练值。在每个有增益的数据集上，删除份额与表 V 相差不超过 0.04，两种归因都把 `har` 与 `adult` 的增益归于删除。

主要经验。实验支持四条经验。第一，干净数据不是清洗的上限，DemandClean 在五个基准数据集上超过它。第二，写回真值的价值随数据集而变，并在同一任务类型内改变符号，因此清洗方法需要逐格判断一个标签是否值得。第三，零成本操作承担了很大一部分增益，值估计写回的真值多于标注预算能支付的数量，在 `har` 上删除几乎独自承担全部增益。第四，在质量状态上做出的决策无需重训即可迁移到同一任务类型的新数据集。

G 补充相关工作

面向机器学习的清洗研究测量数据错误与清洗方法如何影响下游模型。CleanML [3] 在分类任务上比较了针对缺失值、离群值、重复、不一致与错误标签的清洗方法，Mohammed 等 [4] 把六个数据质量维度与常用分类、回归和聚类算法的性能联系起来，Neutatz 等 [5] 考察了 AutoML 搜索在何种情况下会在选出的流水线中包含清洗步骤。DIANA [6] 把这类证据汇集成知识库，向用户推荐数据准备步骤。这些研究确立了数据层正确性与下游增益之间的差异，DemandClean 把这一差异转化为标注预算下逐格决策问题的目标。

References

[1] A. Narayan, I. Chami, L. J. Orr, and C. Ré, "Can foundation models wrangle your data?" *Proc. VLDB Endow.*, vol. 16, no. 4, pp. 738–746, 2022. [Online]. Available: https://www.vldb.org/pvldb/vol16/p738-narayan.pdf

[2] Anthropic, "Introducing Claude Opus 4.8," https://www.anthropic.com/news/claude-opus-4-8, May 2026, list price of the API: $5 per million input tokens and $25 per million output tokens. Accessed Oct. 18, 2026.

[3] P. Li, X. Rao, J. Blase, Y. Zhang, X. Chu, and C. Zhang, "CleanML: A study for evaluating the impact of data cleaning on ML classification tasks," in *Proc. IEEE Int. Conf. Data Eng. (ICDE)*, 2021, pp. 13–24.

[4] S. Mohammed, L. Budach, M. Feuerpfeil, N. Ihde, A. Nathansen, N. Noack, H. Patzlaff, F. Naumann, and H. Harmouch, "The effects of data quality on machine learning performance on tabular data," *Inf. Syst.*, vol. 132, p. 102549, 2025.

[5] F. Neutatz, B. Chen, Y. Alkhatib, J. Ye, and Z. Abedjan, "Data cleaning and AutoML: Would an optimizer choose to clean?" *Datenbank-Spektrum*, vol. 22, no. 2, pp. 121–130, 2022.

[6] C. Sancricca, "DIANA: A knowledge-driven framework for data-centric AI," in *Proc. Workshops EDBT/ICDT Joint Conf.*, ser. CEUR Workshop Proceedings, vol. 3651, 2024.